GRUPO 10 · REQUERIMIENTOS FUNCIONALES

Administración de Roles y Permisos

Módulo: Gestión de usuarios internos (backoffice)

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 10
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aiquaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre el CRUD del catálogo de roles del backoffice y la asignación y revocación de roles a un usuario. La revocación de una asignación es una baja lógica: la asignación permanece registrada y se reactiva si el rol vuelve a otorgarse.

Fuera de alcance: Definición de permisos concretos por rol, verificación de permisos al ejecutar operaciones, jerarquías y aprobación de asignaciones por un superior.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy `/api/proxy/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v1/roles	RF-G10-01	Devuelve el catálogo de roles activos.
POST /api/v1/roles	RF-G10-05	Crea un rol nuevo (dentro de los 4 nombres permitidos).
GET /api/v1/roles/{id}	RF-G10-06	Devuelve el detalle de un rol.
PUT /api/v1/roles/{id}	RF-G10-07	Reemplaza nombre y descripción de un rol.
DELETE /api/v1/roles/{id}	RF-G10-08	Da de baja (soft-delete) un rol del catálogo.
GET /api/v1/usuarios/{id}/roles	RF-G10-02	Lista los roles vigentes de un usuario.
POST /api/v1/usuarios/{id}/roles	RF-G10-03	Asigna un rol a un usuario, o reactiva una asignación revocada.
DELETE /api/v1/usuarios/{id}/roles/{roleId}	RF-G10-04	Revoca un rol de un usuario mediante baja lógica.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key.".
- Cada API key admite un máximo de **200 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-`

`Remaining` y `X-RateLimit-Reset`.

- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` de la API hace baja lógica (`activo = false`), devolviendo `204 No Content` sin cuerpo.
- Casi todas las tablas de negocio (`sesiones`, `transferencias`, `facturas`, `tarjetas`, `notificaciones`, `ordenes`, `reservas`, `movimientos`, `roles`, además de `usuarios` y `cuentas` que ya la tenían) tienen columna `activo`/`activa`; todo `GET`, `PUT` y `DELETE` de esa tabla filtra u opera solo sobre filas con `activo = true` — una fila dada de baja deja de ser visible y de poder reemplazarse, pero no se borra.
- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.
- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- El catálogo de roles solo admite cuatro nombres posibles (`admin`, `soporte`, `auditor`, `operador`, restringidos por una restricción `CHECK`), pero desde el CRUD la API sí permite crear, reemplazar y dar de baja filas del catálogo dentro de esos cuatro nombres.
- Un mismo usuario no puede tener dos asignaciones del mismo rol: la base lo impide con una restricción de unicidad, y la asignación la contempla reactivando la fila existente.

3. Modelo de datos del módulo

roles

Catálogo de roles del backoffice. Desde el CRUD, el módulo también lo escribe.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`nombre`	`text`, **único** (restricción `UNIQUE` de tabla, no parcial), uno de `admin` / `soporte` / `auditor` / `operador`
`descripcion`	`text`, opcional
`created_at`	`timestamptz`, por defecto `now()`
`activo`	`boolean`, obligatorio, por defecto `true`; lo usan `GET/PUT/DELETE` del catálogo

usuario_roles

Asignaciones de rol a usuario. Es la tabla que escribe el módulo.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`role_id`	`bigint`, obligatorio, referencia a `roles(id)`
`activo`	`boolean`, obligatorio, por defecto `true`; es lo que distingue vigente de revocada
`asignado_en`	`timestamptz`, por defecto `now()`; **no** se actualiza al reactivar
Restricción	La combinación de usuario y rol es única: una sola fila por par

> El recurso `roles` de este módulo (permisos de backoffice) no tiene relación con el grupo de curso del alumno, que se consulta por separado con `GET /api/v1/roster?email=`. Son dos conceptos distintos que comparten un nombre parecido.

4. Requerimientos funcionales

RF-G10-01 — Consultar el catálogo de roles `GET /api/v1/roles`

Devuelve todos los roles disponibles para asignar, con su nombre y su descripción. Es también el endpoint más liviano de la API, por lo que la aplicación web lo usa para comprobar que una API key es válida.

Entradas y validaciones

- No recibe parámetros.

Reglas de negocio

- Devuelve únicamente roles con `activo = true`, ordenados por identificador ascendente, sin filtros ni paginación.
- Un rol dado de baja con RF-G10-08 deja de aparecer aquí, aunque sus asignaciones existentes en `usuario_roles` no se ven afectadas.
- Los identificadores de rol que devuelve este endpoint son los que se usan para asignar y revocar en `/usuarios/{id}/roles`.

Respuesta esperada

- `200 OK` con `data` como arreglo de roles, cada uno con `id`, `nombre`, `descripcion` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/roles/route.ts`

RF-G10-02 — Listar los roles vigentes de un usuario

`GET /api/v1/usuarios/{id}/roles`

Devuelve los roles actualmente vigentes de un usuario, con el nombre y la descripción de cada rol.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo del usuario.

Reglas de negocio

- Devuelve únicamente las asignaciones vigentes (`activo = true`): las revocadas quedan fuera del resultado.
- Cada elemento combina los datos de la asignación con el nombre y la descripción del rol.
- El resultado se ordena por identificador de asignación ascendente.
- Un usuario sin roles vigentes, y también un usuario inexistente, devuelven una lista vacía con estado 200: **este endpoint no distingue ambos casos**.

Respuesta esperada

- `200 OK` con `data` como arreglo; cada elemento incluye `id`, `usuario_id`, `role_id`, `activo`, `asignado_en`, `nombre` y `descripcion`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	El identificador de usuario no es un entero positivo.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/usuarios/[id]/roles/route.ts`

RF-G10-03 — Asignar un rol a un usuario

`POST /api/v1/usuarios/{id}/roles`

Otorga un rol del catálogo a un usuario. Si ese rol ya le había sido otorgado y luego revocado, la asignación original se reactiva en lugar de crearse una nueva.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo del usuario.
- `roleId` (obligatorio, en el cuerpo): número entero positivo del rol.

Reglas de negocio

- Si el usuario no tenía ese rol, se crea la asignación con `activo = true`.
- Si ya existía una asignación de ese rol (vigente o revocada), la misma fila se marca como vigente: no se duplica y conserva su identificador original.
- Al reactivar, la fecha de asignación original **no** se actualiza: sigue reflejando el primer otorgamiento.
- **El código de estado distingue alta de reactivación:** `201` si la asignación es nueva, `200` si existía (vigente o revocada) y se reactivó. Internamente se usa el truco `xmax = 0` de Postgres sobre el `RETURNING` del `INSERT ... ON CONFLICT DO UPDATE` para saberlo con certeza, no una heurística.
- Volver a asignar un rol ya vigente es una operación aceptada, con estado 200 y sin efecto observable.
- El usuario y el rol deben existir: si alguno no existe, la operación falla por violación de clave foránea, que se traduce a `400 VALIDATION_ERROR`.
- **No se verifica quién asigna:** cualquier portador de una API key válida puede otorgar el rol `admin` a cualquier usuario.

Respuesta esperada

- `201 Created` con `data` conteniendo la asignación resultante (`id`, `usuario_id`, `role_id`, `activo`, `asignado_en`) cuando la asignación es nueva.
- `200 OK` con el mismo `data` cuando la asignación ya existía y se reactivó (o ya estaba vigente).

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta `roleId`, alguno de los identificadores no es un entero positivo, o el usuario o el rol indicados no existen.

Fuente: `app/api/v1/usuarios/[id]/roles/route.ts`

RF-G10-04 — Revocar un rol de un usuario

`DELETE /api/v1/usuarios/{id}/roles/{roleId}`

Quita un rol a un usuario. La asignación no se borra: se marca como no vigente, de modo que quede registro de que existió.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo del usuario.
- `roleId` (obligatorio, en la ruta): número entero positivo del rol.

Reglas de negocio

- La revocación es una baja lógica: la fila permanece con `activo = false`. El sistema nunca borra físicamente la asignación.
- Solo se revoca la asignación que corresponde exactamente a ese usuario y ese rol, y que esté vigente (`activo = true`).
- Si el par usuario-rol nunca existió, la operación responde 404 con el mensaje "Asignación de rol no encontrada.".
- **Revocar una asignación ya revocada también responde 404, no 204**: la condición `AND activo = true` del `UPDATE` hace que la segunda revocación no encuentre ninguna fila para actualizar. A diferencia de asignar (RF-G10-03), revocar **no es idempotente**.
- La revocación no elimina ningún dato del usuario ni afecta sus otros roles.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una asignación vigente de ese rol para ese usuario (nunca existió, o ya estaba revocada).
`VALIDATION_ERROR`	400	Alguno de los identificadores no es un entero positivo.

Fuente: `app/api/v1/usuarios/[id]/roles/[roleId]/route.ts`

RF-G10-05 — Crear un rol

`POST /api/v1/roles`

Agrega una fila al catálogo de roles, dentro de los cuatro nombres permitidos por la restricción `CHECK`.

Entradas y validaciones

- `nombre` (obligatorio): uno de `admin`, `soporte`, `auditor`, `operador`.
- `descripcion` (opcional): texto libre.

Reglas de negocio

- `nombre` tiene una restricción `UNIQUE` de tabla completa (no parcial por `activo`): si ya existe una fila con ese nombre, **incluso si está dada de baja**, el alta es rechazada por duplicado.
- No hay forma de reemplazar los cuatro roles sembrados de fábrica salvo dándolos de baja primero (y entonces ese nombre queda inutilizable para un alta futura, ver brechas).

Respuesta esperada

- `201 Created` con `data` conteniendo el rol creado.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta `nombre`, o tiene un valor fuera de la lista permitida.
`CONFLICT`	409	Ya existe una fila con ese `nombre` (vigente o dada de baja).

Fuente: `app/api/v1/roles/route.ts`

RF-G10-06 — Consultar un rol

`GET /api/v1/roles/{id}`

Devuelve el detalle de un rol puntual del catálogo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve el rol cuyo identificador coincide exactamente, siempre que siga `activo = true`.
- Si no existe o ya está dado de baja, responde 404 con el mensaje "Rol no encontrado.".

Respuesta esperada

- `200 OK` con `data` conteniendo el rol completo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un rol vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/roles/[id]/route.ts`

RF-G10-07 — Reemplazar un rol

`PUT /api/v1/roles/{id}`

Reemplaza el nombre y la descripción de un rol existente del catálogo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `nombre` (obligatorio): uno de `admin`, `soporte`, `auditor`, `operador`.
- `descripcion` (opcional): texto libre.

Reglas de negocio

- Solo puede reemplazar un rol con `activo = true`; uno dado de baja responde 404 (y no hay forma de reactivarlo por este medio, porque la condición `AND activo = true` del `UPDATE` no encuentra la fila).
- Reemplazar el `nombre` por uno ya usado en otra fila (vigente o de baja) es rechazado por unicidad.

Respuesta esperada

- `200 OK` con `data` conteniendo el rol ya actualizado.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un rol vigente con ese identificador.
`VALIDATION_ERROR`	400	Falta `nombre` o tiene un valor fuera de la lista permitida.
`CONFLICT`	409	El nuevo `nombre` ya pertenece a otra fila del catálogo.

Fuente: `app/api/v1/roles/[id]/route.ts`

RF-G10-08 — Dar de baja un rol

`DELETE /api/v1/roles/{id}`

Da de baja lógica un rol del catálogo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activo = false`; la fila permanece en la base, nunca se borra físicamente.
- No verifica si el rol tiene asignaciones vigentes en `usuario_roles`: un rol con usuarios asignados puede darse de baja igual, y esas asignaciones siguen existiendo (el usuario sigue viendo el rol en RF-G10-02, aunque el rol ya no aparezca en el catálogo de RF-G10-01).
- **No existe forma de reactivar un rol dado de baja** (ni `PUT` ni ningún otro endpoint lo permite), y su `nombre` queda inutilizable para un alta futura por la restricción `UNIQUE` no parcial.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un rol vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/roles/[id]/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.
- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body.".
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`, ej. un email o número de factura repetido) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos cae en `400 EXECUTION_ERROR`.

- `DELETE` **es siempre baja lógica:** actualiza `activo = false` (o `activa` en `cuentas`) y responde `204 No Content` **sin cuerpo** — no `200` con la fila afectada.
- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera Next.js automáticamente, ninguna ruta lo arma a mano, y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear`, `activar`, `confirmar`, `cancelar`, `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.
- **Las transiciones de estado anteriores al CRUD no miran `activo`:** `PATCH .../bloquear`, `.../activar`, `.../leer`, `.../confirmar`, `.../cancelar`, `.../kyc` y `POST .../pagar` no filtran por `activo = true` (ninguno de esos archivos se tocó al agregar el CRUD). Un recurso dado de baja con el nuevo `DELETE` deja de aparecer en `GET`, pero **sigue pudiendo bloquearse, confirmarse, leerse o pagarse** a través de su acción dedicada — una inconsistencia real del sandbox, no un comportamiento a asumir como intencional.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso; también un POST de upsert que reactivó una fila existente en vez de crearla.
`201 Created`	POST que crea una fila nueva.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Falta el header `x-api-key`, o la clave no existe o está inactiva.
`RATE_LIMITED`	429	Se superaron las 200 requests por minuto de esa API key.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe, ya está dado de baja (`activo=false`), o no está en un estado que admita la operación.
`CONFLICT`	409	Violación de restricción `UNIQUE` en Postgres (ej. email, número de factura o nombre de rol duplicado).
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local. **La interfaz web todavía no tiene pantallas para `PUT` / `DELETE` ni para los recursos nuevos (`sesiones` , `movimientos`)**: solo cubre los flujos de creación y consulta que ya existían; esos requerimientos se prueban llamando la API directamente (Postman, `curl`, o el cliente HTTP del framework de automatización), no navegando la web.

Pantalla	Qué permite hacer
`/roles` — Roles del usuario	Tabla con el catálogo de roles y, para cada uno, si está vigente para el usuario en sesión, con las acciones de asignar y revocar. Ejecuta los cuatro requerimientos del módulo.

- La pantalla opera siempre sobre el usuario que está en sesión: no hay selector de usuario, por lo que probar la asignación a otro usuario requiere llamar la API directamente.
- Después de asignar o revocar, la pantalla vuelve a consultar el estado y refleja el cambio sin recargar.
- El catálogo de roles es además la comprobación que hace la pantalla de acceso para validar una API key.
- No hay pantalla para el CRUD del catálogo: `POST /roles` y `GET` / `PUT` / `DELETE /roles/{id}` (RF-G10-05 a 08) solo se pueden ejercitar llamando la API directamente.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G10-01 — Consultar el catálogo de roles

- Al consultar el catálogo con una API key válida, la respuesta es 200 y `data` contiene los cuatro roles del sandbox.
- Los nombres devueltos pertenecen al conjunto `admin`, `soporte`, `auditor` y `operador`.
- Al consultar el catálogo sin API key, la respuesta es 401 `UNAUTHORIZED`; este es el mismo control que usa la pantalla de acceso de la web para validar la clave.
- Consultas sucesivas devuelven el mismo catálogo: no hay endpoint que lo modifique.

RF-G10-02 — Listar los roles vigentes de un usuario

- Dado un usuario con un rol asignado, al listar sus roles la respuesta es 200 y el rol figura con `activo` en verdadero y su `nombre` resuelto.
- Tras revocar ese rol, el listado deja de incluirlo.
- Al listar los roles de un usuario inexistente, la respuesta es 200 con `data` vacío, no un 404.
- Al listar con un identificador no numérico, la respuesta es 400 `VALIDATION_ERROR`.

RF-G10-03 — Asignar un rol a un usuario

- Al asignar un rol nuevo a un usuario, la respuesta es 201 y `data.activo` es verdadero.

- Al asignar dos veces el mismo rol al mismo usuario, la segunda respuesta es 200, no 201, y el listado sigue mostrando una única asignación de ese rol.
- Al revocar un rol y volver a asignarlo, la respuesta es 200 (reactivación) y la asignación conserva el mismo `id` que tenía antes de la revocación.
- Tras esa reactivación, `asignado_en` conserva la fecha del primer otorgamiento.
- Al asignar un `roleId` inexistente, la respuesta es 400 `VALIDATION_ERROR` y no se crea ninguna asignación.
- Al asignar un rol a un usuario inexistente, la respuesta es 400 `VALIDATION_ERROR`.

RF-G10-04 — Revocar un rol de un usuario

- Dado un usuario con un rol vigente, al revocarlo la respuesta es 204 sin cuerpo.
- Tras la revocación, el rol ya no aparece en el listado de roles vigentes del usuario.
- Al revocar dos veces el mismo rol, la segunda respuesta es 404 `NOT_FOUND`, no 204: revocar no es idempotente.
- Al revocar un rol que el usuario nunca tuvo, la respuesta es 404 `NOT_FOUND` con el mensaje "Asignación de rol no encontrada.".
- Revocar un rol no afecta a los demás roles vigentes del usuario.

RF-G10-05 — Crear un rol

- Si los cuatro nombres del catálogo sembrado siguen vigentes, crear cualquiera de ellos de nuevo responde 409 `CONFLICT`.
- Al crear con `nombre=supervisor`, la respuesta es 400 `VALIDATION_ERROR`.
- El rol creado aparece de inmediato en el listado de RF-G10-01.

RF-G10-06 — Consultar un rol

- Dado un rol del catálogo sembrado, al consultarlo por su `id` la respuesta es 200.
- Al consultar un rol dado de baja, la respuesta es 404, aunque siga existiendo como fila en la base.

RF-G10-07 — Reemplazar un rol

- Dado un rol existente, al reemplazar su `descripcion` la respuesta es 200 y el cambio se refleja al consultarlo.
- Al reemplazar el `nombre` por el de otro rol existente, la respuesta es 409 `CONFLICT`.
- Al reemplazar un rol dado de baja, la respuesta es 404 `NOT_FOUND`: `PUT` no sirve para reactivarlo.

RF-G10-08 — Dar de baja un rol

- Dado un rol vigente sin usuarios asignados, al darlo de baja la respuesta es 204 sin cuerpo.
- Tras la baja, `GET /roles/{id}` responde 404 y el rol deja de aparecer en RF-G10-01.
- Dar de baja un rol con usuarios asignados no quita el rol del listado de roles vigentes de esos usuarios (RF-G10-02).
- Tras dar de baja el rol `operador`, crear un rol nuevo con `nombre=operador` responde 409 `CONFLICT`: el nombre queda inutilizable.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Marco del BCP sobre control interno y seguridad de la información en entidades financieras	Los accesos privilegiados deben otorgarse bajo aprobación, con segregación de funciones y revisión periódica.	No implementado: cualquier portador de una API key válida puede otorgarse o quitar cualquier rol, sin aprobación ni segregación.
Principio de mínimo privilegio	Cada usuario debe tener únicamente los permisos necesarios para su función.	No implementado: los roles son etiquetas; ninguna operación del sandbox verifica el rol del usuario antes de ejecutarse.
Trazabilidad de altas y bajas de accesos	Debe quedar registro de quién otorgó o revocó cada permiso y cuándo, y el estado de éxito de la operación debe ser semánticamente correcto.	Parcial: la baja lógica conserva la asignación y toda request queda auditada, la asignación no guarda quién la ejecutó y la fecha no se actualiza al reactivarla, pero desde esta actualización sí se distingue correctamente alta (201) de reactivación (200), y la revocación devuelve el 204 estándar.
Ley 6534/2020 de protección de datos personales	El acceso a datos personales debe estar restringido según el rol del funcionario.	No implementado: todos los endpoints exponen los mismos datos a cualquier API key, con independencia de los roles asignados.
Revisión periódica de accesos (recertificación)	Los permisos vigentes deben revisarse periódicamente y revocarse los innecesarios.	Parcial: el listado de roles vigentes permite la revisión, pero no hay vencimiento ni proceso de recertificación.

Brechas conocidas del sandbox

- Los roles no otorgan ni restringen nada: ningún endpoint verifica el rol del usuario antes de operar.
- Cualquier API key válida puede asignar el rol `admin` a cualquier usuario, incluido a sí mismo.
- El listado de roles de un usuario inexistente devuelve una lista vacía con estado 200, en lugar de 404.
- La asignación no registra quién la realizó, y al reactivar una asignación revocada no se actualiza la fecha de otorgamiento.
- El catálogo de roles sigue sin un modelo de permisos asociado: crear un rol nuevo no le da ningún efecto sobre las operaciones de la API.
- No hay vencimiento de asignaciones ni proceso de recertificación de accesos.
- Un rol dado de baja no puede reactivarse por ningún medio, y su `nombre` queda inutilizable para siempre porque la restricción `UNIQUE` de `roles.nombre` no es parcial (no ignora las filas con `activo = false`).
- Dar de baja un rol no afecta ni notifica a los usuarios que ya lo tenían asignado: siguen viéndolo en `GET /usuarios/{id}/roles` aunque haya desaparecido del catálogo.

- A diferencia de la mayoría de los `DELETE` del sandbox, revocar una asignación de rol (RF-G10-04) no es idempotente: repetirlo da 404, no 204.